МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа № *2* по курсу «Операционные системы и архитектура компьютера»

Выполнил: Ф. А. Алексеев
Группа: М8О-204БВ-25
Преподаватель: Н. А. Гапонов
Дата: 22.10.2026
Оценка: _____

Москва, 2026

Условие

Цель работы: получение практических навыков в создании и управлении процессами в ОС и обеспечении передачи данных между ними через каналы.

Задание: составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними. Основной процесс должен создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Вариант 4: Пользователь вводит команды вида «число число число<endline>». Эти числа передаются от родительского процесса в дочерний. Дочерний процесс производит деление первого числа на последующие, а результат выводит в файл. Если происходит деление на 0, то дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип float. Количество чисел может быть произвольным..

Метод решения

Программа состоит из двух отдельных исполняемых файлов: parent.exe и child.exe. Работа выполнена на ОС Windows. Процессы обмениваются данными через два канала — первый используется для записи родительским процессом и чтением дочернего, второй — для записи дочернего процесса в стандартный вывод (stdout и stderr). Родительский процесс передает строку, которую дочерний разбивает и обрабатывает для получения результата последовательного деления, а также проверки деления на ноль. Оба процесса возвращают 3 различных кода: 0 — при успехе, 2 — при обнаружении деления на ноль,1 — иные ошибки завершения процесса (с описанием в stderr).

Описание программы

Исходный код программы разделён на два файла. В проекте приведен файл для автоматической сборки проекта через CMake. При запуске родительского процесса на ввод подается имя файла для записи результата, затем произвольное количество строк с произвольным количеством чисел для вычислений. Полный код приведён в приложении А.

Структура файлов

Таблица 1 – Файлы проекта

Файл	Назначение
`parent.c`	родительский процесс — читает имя файла и строки с числами, создаёт каналы и дочерний процесс, пересылает данные, определяет результат по коду завершения ребёнка
`child.c`	дочерний процесс — читает числа из stdin, выполняет деление, проверяет деление на 0, пишет результат в файл, отвечает родителю в stdout
`CMakeLists.txt`	Общая сборка проекта через CMake

Основные типы данных и функции

- `SECURITY_ATTRIBUTES` с флагом наследования — для создания каналов, доступных ребёнку
- `STARTUPINFO` с полями `hStdInput` и `hStdOutput` — для замены на стандартные потоки ребёнка

- `PROCESS_INFORMATION` — получение дескрипторов и идентификаторов созданного процесса.
- `int parse_number(char **p, float *out)` — берёт следующее число из строки и сдвигает указатель за него. Возвращает 1, если число прочитано, 0, если числа закончились, и -1, если слово не является числом после проверки.
- `int process_line(char *line, FILE *f)` — обрабатывает одну строку: читает первое число, делит на остальные, при нулевом делителе возвращает 2, при ошибке разбора возвращает 1, иначе записывает результат в файл и возвращает 0.
- `int main(int argc, char *argv[])` — открывает файл, в цикле читает строки из stdin, вызывает process_line, отправляет ответ в stdout и формирует код завершения.

Используемые системные вызовы

Таблица 2 – Используемые системные вызовы

Системный вызов	Назначение в программе
`CreatePipe()`	Создание анонимного канала
`CreateProcess()`	Создание дочернего процесса
`WriteFile()`	Запись строки с числами в канал
`ReadFile()`	Чтение ответа ребёнка из канала
`CloseHandle()`	Закрытие концов каналов и дескрипторов процесса
`SetHandleInformation()`	Параметр наследования родительских концов каналов дочерним процессом
`DWORD WaitForSingleObject()`	Ожидание завершения дочернего процесса
`int GetExitCodeProcess()`	Получение кода завершения дочернего процесса
`HANDLE GetStdHandle()`	Получение дескриптора стандартного потока (потока ошибок) для передачи ребёнку

Ключевые фрагменты кода

Листинг 1 – Создание каналов (parent.c)

```
SECURITY_ATTRIBUTES sa = { sizeof(sa), NULL, TRUE };
HANDLE r1, w1, r2, w2;

# …

if (!CreatePipe(&r1, &w1, &sa, 0) || !CreatePipe(&r2, &w2, &sa, 0)) {
 fprintf(stderr, "CreatePipe error %lu\n", GetLastError());
 return 1;
}
if (!SetHandleInformation(w1, HANDLE_FLAG_INHERIT, 0) ||
 !SetHandleInformation(r2, HANDLE_FLAG_INHERIT, 0)) {
 fprintf(stderr, "SetHandleInformation error %lu\n", GetLastError());
 return 1;
}
```

Листинг 2 – Создание дочернего процесса (parent.c)

```
si.hStdInput  = r1;
si.hStdOutput = w2;
si.hStdError  = GetStdHandle(STD_ERROR_HANDLE);

if (!CreateProcess(NULL, cmd, NULL, NULL, TRUE, 0, NULL, NULL, &si, &pi)) {
      fprintf(stderr, "CreateProcess error %lu\n", GetLastError());
      return 1;
}

#…

CloseHandle(r1);
CloseHandle(w2);
```

Листинг 3 – Деление с проверкой на 0 и запись в файл (child.c)

```
while (1) {
 r = parse_number(&p, &x);
 if (r == 0) break;
 if (r < 0) {
      fprintf(stderr, "Invalid number\n");
      return 1;
 }
 if (x == 0) return 2;
 result /= x;
 if (fprintf(f, "%g\n", result) < 0 || fflush(f) != 0) {
      perror("W");
      return 1;
 }
}
```

Листинг 4 – Обработка кода завершения (parent.c)

```
DWORD code;
if (!GetExitCodeProcess(pi.hProcess, &code)) {
      fprintf(stderr, "GetExitCodeProcess error%lu\n", GetLastError());
      return 1;
}
if (code == 0) return code;
else if (code == 2) {
      printf("Division by 0 error\n");
} else {
      printf("Child finished with code %lu\n", code);
}
return code;
```

Результаты

Программа принимает на ввод произвольное количество строк с произвольным количеством чисел.При возникновении ошибок данные записываются в stderr, а процесс, в котором возникла ошибка, возвращает её код (1 или 2). Ниже представлены тесты для демонстрации работы программы при разных входных данных.

Стандартный тест:

stdin:

```
> parent.exe
Enter filename: file.txt
10 5 2 2
4 -3
^Z
```

file.txt:

```
0.5
-1.33333
```

Деление на 0:

stdin:

```
> parent.exe
Enter filename: file.txt
10 5
10 0
^Z
Division by 0 error
```

file.txt:

```
2
```

Корректность ввода чисел:

stdin:

```
> parent.exe
Enter filename: file.txt
10 5
10 abc
Invalid number
^Z
Child finished with code 1
```

file.txt:

```
2
```

Демонстрация работы Process Monitor

Программа запущена под управлением Process Monitor (ProcMon) с фильтром по имени процесса parent.exe или child.exe. Полный вывод приведен в приложении Б.

Создание дочернего процесса из ProcMon:

```
13:48:51     parent.exe           CreateFile     C:\...\src\build\child.exe   SUCCESS
13:48:51     parent.exe           Process Create :\...\src\build\child.exe    SUCCESS
      PID: 25752, Command line: child.exe "file.txt"
```

Завершение по окончании чтения результата:

```
13:48:55     parent.exe           CloseFile      C:\...\src\build\child.exe   SUCCESS
13:48:56     parent.exe           CloseFile      C:\...\src\build\file.txt    SUCCESS
13:48:56     parent.exe           CloseFile      C:\...\src\    SUCCESS
```

Выводы

В ходе работы реализованы две программы на Си: родительская и дочерняя, которые взаимодействуют через два канала. Стандартный поток дочернего процесса перенаправляется через канал. Освоены создание процесса с подменой стандартных потоков, обмен данными через каналы, ожидание завершения процесса и получение его кода возврата. Корректность подтверждается тестами: на допустимых данных результаты записываются в файл, при делении на 0 оба процесса завершаются с кодом 2, при неверном вводе ребёнок возвращает код 1.

Приложение А. Исходный код программы

Листинг А.1 – Файл parent.c

```
#include <windows.h>
#include <stdio.h>
#include <string.h>

#define MAX_LINE 1024

int main(void) {
    char name[256];
    char cmd[300];
    char line[MAX_LINE + 2];

    printf("Enter filename: ");
    if (!fgets(name, sizeof(name), stdin)) {
        fprintf(stderr, "Couldn't read filename\n");
        return 1;
    }
    name[strcspn(name, "\r\n")] = '\0';
    if (name[0] == '\0') {
        fprintf(stderr, "Filename is empty\n");
        return 1;
    }

    if (snprintf(cmd, sizeof(cmd), "child.exe \"%s\"", name) >= sizeof(cmd)) {
        fprintf(stderr, "Filename length is out of range\n");
        return 1;
    }

    SECURITY_ATTRIBUTES sa = { sizeof(sa), NULL, TRUE };
    HANDLE r1, w1;
    HANDLE r2, w2;

    if (!CreatePipe(&r1, &w1, &sa, 0) || !CreatePipe(&r2, &w2, &sa, 0)) {
        fprintf(stderr, "CreatePipe error %lu\n", GetLastError());
        return 1;
    }

    if (!SetHandleInformation(w1, HANDLE_FLAG_INHERIT, 0) || !SetHandleInformation(r2,
HANDLE_FLAG_INHERIT, 0)) {
        fprintf(stderr, "SetHandleInformation error %lu\n", GetLastError());
        return 1;
    }

    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    si.dwFlags = STARTF_USESTDHANDLES;
    si.hStdInput  = r1;
    si.hStdOutput = w2;
```

```
    si.hStdError  = GetStdHandle(STD_ERROR_HANDLE);

    if (!CreateProcess(NULL, cmd, NULL, NULL, TRUE, 0, NULL, NULL, &si, &pi)) {
        fprintf(stderr, "CreateProcess error %lu\n", GetLastError());
        return 1;
    }

    CloseHandle(r1);
    CloseHandle(w2);

    while (fgets(line, MAX_LINE, stdin) != NULL) {
        size_t len = strlen(line);

        if (line[len - 1] != '\n') {
            if (!feof(stdin)) {
                int ch;
                while ((ch = getchar()) != '\n' && ch != EOF) { }
                fprintf(stderr, "Line length is out of range\n");
                continue;
            }
            line[len++] = '\n';
            line[len] = '\0';
        }

        DWORD n;
        if (!WriteFile(w1, line, (DWORD)len, &n, NULL) || n != len) {
            break;
        }
    }

    CloseHandle(w1);
    CloseHandle(r2);

    if (WaitForSingleObject(pi.hProcess, INFINITE) == WAIT_FAILED) {
        fprintf(stderr, "WaitForSingleObject error %lu\n", GetLastError());
        return 1;
    }

    DWORD code;
    if (!GetExitCodeProcess(pi.hProcess, &code)) {
        fprintf(stderr, "GetExitCodeProcess error %lu\n", GetLastError());
        return 1;
    }
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);

    if (code == 0) {
        return code;
    } else if (code == 2) {
        printf("Division by 0 error\n");
    } else {
        printf("Child finished with code %lu\n", code);
```

```
    }
    return code;
}
```

Листинг А.2 – Файл child.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <errno.h>

#define MAX_LINE 1024

static int parse_number(char **p, float *out) {
    while (isspace(**p)) (*p)++;
    if (**p == '\0') return 0;

    char *end;
    errno = 0;
    float x = strtof(*p, &end);

    if (end == *p) return -1;
    if (errno == ERANGE) return -1;
    if (*end != '\0' && !isspace(*end)) return -1;

    *out = x;
    *p = end;
    return 1;
}

static int process_line(char *line, FILE *f) {
    char *p = line;
    float result, x;

    int r = parse_number(&p, &result);
    if (r == 0) return 0;
    if (r < 0) {
        fprintf(stderr, "Invalid number\n");
        return 1;
    }

    while (1) {
        r = parse_number(&p, &x);
        if (r == 0) break;
        if (r < 0) {
            fprintf(stderr, "Invalid number\n");
            return 1;
        }
        if (x == 0) return 2;
        result /= x;
    }
```

```
    if (fprintf(f, "%g\n", result) < 0 || fflush(f) != 0) {
        perror("W");
        return 1;
    }
    return 0;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Invalid argument amount. Requires a filename for output\n");
        return 1;
    }

    FILE *f = fopen(argv[1], "w");
    if (f == NULL) {
        perror("fopen");
        return 1;
    }

    int code = 0;
    char line[MAX_LINE + 2];

    while (fgets(line, sizeof(line), stdin) != NULL) {
        int r = process_line(line, f);
        if (r != 0) {
            code = r;
            break;
        }
    }

    if (fclose(f) != 0) {
        perror("fclose");
        if (code == 0) code = 1;
    }
    return code;
}
```

Листинг А.3 – Файл CMakeLists.txt

```
cmake_minimum_required(VERSION 4.2.3)
project(lab2 LANGUAGES C)
set(CMAKE_C_STANDARD 11)

add_executable(parent parent.c)
add_executable(child child.c)
```

Приложение Б. Полный вывод Process Monitor

Листинг Б.1 – Вывод Process Monitor при исполнении программы

```
14:03:37,8899837    parent.exe    24776  Process Start        SUCCESS Parent PID:
20452, Command line: "C:\Users\fedor\OneDrive\Документы\os_lab2\src\build\parent.exe",
Current directory: C:\Users\fedor\OneDrive\Документы\os_lab2\src\
14:03:37,8928762    parent.exe    24776  CreateFile    C:\Windows\Prefetch\PARENT.EXE-
57ADF4C2.pf  SUCCESS Desired Access: Generic Read, Disposition: Open, Options:
Synchronous IO Non-Alert, Attributes: n/a, ShareMode: None, AllocationSize: n/a,
OpenResult: Opened
14:03:37,8936170    parent.exe    24776  CloseFile     C:\Windows\Prefetch\PARENT.EXE-
57ADF4C2.pf  SUCCESS
14:03:37,8955604    parent.exe    24776  CreateFile    C:\Users\fedor\OneDrive\
Документы\os_lab2\src      SUCCESS Desired Access: Execute/Traverse, Synchronize,
Disposition: Open, Options: Directory, Synchronous IO Non-Alert, Attributes: n/a,
ShareMode: Read, Write, AllocationSize: n/a, OpenResult: Opened
14:03:41,2473715    parent.exe    24776  CreateFile    C:\Users\fedor\OneDrive\
Документы\os_lab2\src\build\child.exe     SUCCESS Desired Access: Read Attributes,
Disposition: Open, Options: Open Reparse Point, Attributes: n/a, ShareMode: Read, Write,
Delete, AllocationSize: n/a, OpenResult: Opened
14:03:41,2474199    parent.exe    24776  CloseFile     C:\Users\fedor\OneDrive\
Документы\os_lab2\src\build\child.exe     SUCCESS
14:03:41,2476574    parent.exe    24776  CreateFile    C:\Users\fedor\OneDrive\
Документы\os_lab2\src\build\child.exe     SUCCESS Desired Access: Read Attributes,
Disposition: Open, Options: Open Reparse Point, Attributes: n/a, ShareMode: Read, Write,
Delete, AllocationSize: n/a, OpenResult: Opened
14:03:41,2476891    parent.exe    24776  CloseFile     C:\Users\fedor\OneDrive\
Документы\os_lab2\src\build\child.exe     SUCCESS
14:03:41,2481742    parent.exe    24776  CreateFile    C:\Users\fedor\OneDrive\
Документы\os_lab2\src\build\child.exe     SUCCESS Desired Access: Read Data/List
Directory, Execute/Traverse, Read Attributes, Synchronize, Disposition: Open, Options:
Synchronous IO Non-Alert, Non-Directory File, Attributes: N, ShareMode: Read, Delete,
AllocationSize: n/a, OpenResult: Opened
14:03:41,2509319    child.exe     25420  Process Start        SUCCESS Parent PID:
24776, Command line: child.exe "file.txt", Current directory: C:\Users\fedor\OneDrive\
Документы\os_lab2\src\
14:03:41,2512674    parent.exe    24776  CreateFile    C:\Windows\apppatch\sysmain.sdb
     SUCCESS Desired Access: Read Attributes, Disposition: Open, Options: Open For
Backup, Open Reparse Point, Attributes: n/a, ShareMode: Read, Write, Delete,
AllocationSize: n/a, OpenResult: Opened
14:03:41,2514590    parent.exe    24776  CloseFile     C:\Windows\apppatch\sysmain.sdb
     SUCCESS
14:03:41,2516175    parent.exe    24776  CreateFile    C:\Windows\apppatch\
MergeSdbFilesSource\sysMerge.sdb   SUCCESS Desired Access: Read Attributes, Disposition:
Open, Options: Open For Backup, Open Reparse Point, Attributes: n/a, ShareMode: Read,
Write, Delete, AllocationSize: n/a, OpenResult: Opened
14:03:41,2516437    parent.exe    24776  CloseFile     C:\Windows\apppatch\
MergeSdbFilesSource\sysMerge.sdb   SUCCESS
14:03:41,2517497    parent.exe    24776  CreateFile    C:\Windows\apppatch\
01DD5F0E177700B5.sysmain.sdb       SUCCESS Desired Access: Read Attributes, Disposition:
Open, Options: Open For Backup, Open Reparse Point, Attributes: n/a, ShareMode: Read,
Write, Delete, AllocationSize: n/a, OpenResult: Opened
```

```
14:03:41,2517692    parent.exe    24776  CloseFile     C:\Windows\apppatch\
01DD5F0E177700B5.sysmain.sdb      SUCCESS
14:03:41,2518219    parent.exe    24776  CreateFile    C:\Windows\apppatch\sysmain.sdb
     SUCCESS Desired Access: Read Attributes, Disposition: Open, Options: Open For
Backup, Open Reparse Point, Attributes: n/a, ShareMode: Read, Write, Delete,
AllocationSize: n/a, OpenResult: Opened
14:03:41,2518463    parent.exe    24776  CloseFile     C:\Windows\apppatch\sysmain.sdb
     SUCCESS
14:03:41,2519464    parent.exe    24776  CreateFile    C:\Windows\apppatch\
MergeSdbFilesSource\sysMerge.sdb  SUCCESS Desired Access: Read Attributes, Disposition:
Open, Options: Open For Backup, Open Reparse Point, Attributes: n/a, ShareMode: Read,
Write, Delete, AllocationSize: n/a, OpenResult: Opened
14:03:41,2519688    parent.exe    24776  CloseFile     C:\Windows\apppatch\
MergeSdbFilesSource\sysMerge.sdb  SUCCESS
14:03:41,2520244    parent.exe    24776  CreateFile    C:\Windows\apppatch\
01DD5F0E177700B5.sysmain.sdb      SUCCESS Desired Access: Read Attributes, Disposition:
Open, Options: Open For Backup, Open Reparse Point, Attributes: n/a, ShareMode: Read,
Write, Delete, AllocationSize: n/a, OpenResult: Opened
14:03:41,2520384    parent.exe    24776  CloseFile     C:\Windows\apppatch\
01DD5F0E177700B5.sysmain.sdb      SUCCESS
14:03:41,2533322    parent.exe    24776  CreateFile    C:\Windows\apppatch\sysmain.sdb
     SUCCESS Desired Access: Read Attributes, Disposition: Open, Options: Open For
Backup, Open Reparse Point, Attributes: n/a, ShareMode: Read, Write, Delete,
AllocationSize: n/a, OpenResult: Opened
14:03:41,2533625    parent.exe    24776  CloseFile     C:\Windows\apppatch\sysmain.sdb
     SUCCESS
14:03:41,2535361    parent.exe    24776  CreateFile    C:\Windows\apppatch\
MergeSdbFilesSource\sysMerge.sdb  SUCCESS Desired Access: Read Attributes, Disposition:
Open, Options: Open For Backup, Open Reparse Point, Attributes: n/a, ShareMode: Read,
Write, Delete, AllocationSize: n/a, OpenResult: Opened
14:03:41,2535606    parent.exe    24776  CloseFile     C:\Windows\apppatch\
MergeSdbFilesSource\sysMerge.sdb  SUCCESS
14:03:41,2536762    parent.exe    24776  CreateFile    C:\Windows\apppatch\
01DD5F0E177700B5.sysmain.sdb      SUCCESS Desired Access: Read Attributes, Disposition:
Open, Options: Open For Backup, Open Reparse Point, Attributes: n/a, ShareMode: Read,
Write, Delete, AllocationSize: n/a, OpenResult: Opened
14:03:41,2536938    parent.exe    24776  CloseFile     C:\Windows\apppatch\
01DD5F0E177700B5.sysmain.sdb      SUCCESS
14:03:41,2537467    parent.exe    24776  CreateFile    C:\Windows\apppatch\sysmain.sdb
     SUCCESS Desired Access: Read Attributes, Disposition: Open, Options: Open For
Backup, Open Reparse Point, Attributes: n/a, ShareMode: Read, Write, Delete,
AllocationSize: n/a, OpenResult: Opened
14:03:41,2537659    parent.exe    24776  CloseFile     C:\Windows\apppatch\sysmain.sdb
     SUCCESS
14:03:41,2538523    parent.exe    24776  CreateFile    C:\Windows\apppatch\
MergeSdbFilesSource\sysMerge.sdb  SUCCESS Desired Access: Read Attributes, Disposition:
Open, Options: Open For Backup, Open Reparse Point, Attributes: n/a, ShareMode: Read,
Write, Delete, AllocationSize: n/a, OpenResult: Opened
14:03:41,2538761    parent.exe    24776  CloseFile     C:\Windows\apppatch\
MergeSdbFilesSource\sysMerge.sdb  SUCCESS
14:03:41,2539295    parent.exe    24776  CreateFile    C:\Windows\apppatch\
01DD5F0E177700B5.sysmain.sdb      SUCCESS Desired Access: Read Attributes, Disposition:
```

```
Open, Options: Open For Backup, Open Reparse Point, Attributes: n/a, ShareMode: Read,
Write, Delete, AllocationSize: n/a, OpenResult: Opened
14:03:41,2539439    parent.exe     24776  CloseFile      C:\Windows\apppatch\
01DD5F0E177700B5.sysmain.sdb       SUCCESS
14:03:41,2539991    parent.exe     24776  CloseFile      C:\Users\fedor\OneDrive\
Документы\os_lab2\src\build\child.exe     SUCCESS
14:03:41,2541792    child.exe      25420  CreateFile     C:\Windows\Prefetch\CHILD.EXE-
16F50CC4.pf  SUCCESS Desired Access: Generic Read, Disposition: Open, Options:
Synchronous IO Non-Alert, Attributes: n/a, ShareMode: None, AllocationSize: n/a,
OpenResult: Opened
14:03:41,2549719    child.exe      25420  CloseFile      C:\Windows\Prefetch\CHILD.EXE-
16F50CC4.pf  SUCCESS
14:03:41,2571590    child.exe      25420  CreateFile     C:\Users\fedor\OneDrive\
Документы\os_lab2\src       SUCCESS Desired Access: Execute/Traverse, Synchronize,
Disposition: Open, Options: Directory, Synchronous IO Non-Alert, Attributes: n/a,
ShareMode: Read, Write, AllocationSize: n/a, OpenResult: Opened
14:03:41,2589520    child.exe      25420  CreateFile     C:\Users\fedor\OneDrive\
Документы\os_lab2\src\file.txt     SUCCESS Desired Access: Generic Write, Read
Attributes, Disposition: OverwriteIf, Options: Synchronous IO Non-Alert, Non-Directory
File, Attributes: N, ShareMode: Read, Write, AllocationSize: 0, OpenResult: Overwritten
14:03:51,8037915    child.exe      25420  CloseFile      C:\Users\fedor\OneDrive\
Документы\os_lab2\src\file.txt     SUCCESS
14:03:51,8043457    parent.exe     24776  Process Exit           SUCCESS Exit Status: -
1073741510, User Time: 0.0000000 seconds, Kernel Time: 0.0000000 seconds, Private Bytes:
643 072, Peak Private Bytes: 651 264, Working Set: 5 136 384, Peak Working Set:
5 136 384
14:03:51,8043581    child.exe      25420  Process Exit           SUCCESS Exit Status: -
1073741510, User Time: 0.0000000 seconds, Kernel Time: 0.0000000 seconds, Private Bytes:
598 016, Peak Private Bytes: 606 208, Working Set: 4 792 320, Peak Working Set:
4 792 320
14:03:51,8044454    child.exe      25420  CloseFile      C:\Users\fedor\OneDrive\
Документы\os_lab2\src       SUCCESS
14:03:51,8044458    parent.exe     24776  CloseFile      C:\Users\fedor\OneDrive\
Документы\os_lab2\src       SUCCESS
```